R1.04 - Architecture et Systèmes d'Exploitation : Gestion des Processus et Signaux POSIX

Thème principal : Systèmes d'Exploitation et Administration Linux

Synthèse exécutive : Ce cours aborde l'architecture interne des systèmes d'exploitation Unix/Linux en étudiant le concept fondamental de processus. Il explicite la transition d'un programme statique stocké sur le disque dur vers un processus dynamique s'exécutant en mémoire RAM. Afin de permettre l'exécution simultanée de multiples applications sur un nombre limité de cœurs processeurs, le noyau Linux utilise le multitâche préemptif et gère le cycle de vie des processus à travers trois états fondamentaux (Élu, Prêt, Bloqué). La séance décortique le bloc de contrôle de processus (PCB), la structure généalogique parent-enfant (PID/PPID) issue du processus initial systemd/init (PID 1), le mécanisme de création par les appels système fork() et exec(), ainsi que la communication asynchrone et le contrôle des tâches au moyen des signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

Du Programme Statique au Processus Dynamique : Cycle de Vie et Attributs

Pour qu'un ordinateur exécute des instructions, le système d'exploitation doit faire basculer le programme (données passives sur disque) vers l'état de processus (entité active en mémoire RAM). L'ordonnanceur du noyau orchestre cette activité via un modèle à trois états fondamentaux. Lorsqu'un processus est à l'état 'Élu', il occupe physiquement un cœur de processeur. Si son quantum de temps CPU expire, le noyau le bascule à l'état 'Prêt' et attribue le CPU à un autre processus. Si le processus déclenche une lecture sur disque ou attend une saisie clavier, il bascule à l'état 'Bloqué' afin de ne pas gaspiller inutilement des cycles CPU. Dès que la ressource demandée devient disponible, le noyau bascule le processus à l'état 'Prêt' (et non directement à l'état 'Élu'), le replaçant dans la file d'attente de l'ordonnanceur.

- Programme vs Processus : Un programme est un fichier exécutable statique sur disque, tandis qu'un processus est une instance dynamique en RAM.
- Multitâche Préemptif : Découpage du temps CPU en quanta pour simuler l'exécution simultanée des tâches.
- PCB (Process Control Block) : Structure interne du noyau stockant l'état, les registres, la mémoire et l'identifiant PID d'un processus.
- Modèle à 3 États : Élu (Running), Prêt (Ready) et Bloqué (Waiting).

Cas concret enseignant : *L'analogie du livre de recettes de cuisine : Le programme stocké sur le disque dur est comparé à un livre de recettes sagement rangé dans une bibliothèque (passif, statique, ne consommant ni électricité ni ingrédients). Le processus est le cuisinier en train d'exécuter activement la recette dans sa cuisine (dynamique, occupant de la place en RAM sur le plan de travail, consommant du temps CPU lors de la cuisson et manipulant des ustensiles équivalant aux descripteurs de fichiers).*

Filiation, Arborescence et Mécanismes de Création (fork et exec)

Sous Unix, aucun processus ne peut apparaître de façon isolée ou spontanée. Tous les processus s'inscrivent dans un arbre généalogique strict. Lors du démarrage du système d'exploitation, le noyau initialise le premier processus utilisateur nommé systemd (ou init), recevant impérativement le PID 1. Lorsqu'un utilisateur lance une commande dans une console Bash, le shell interactif (processus parent) exécute la création de la tâche en deux temps : d'abord fork() pour cloner le processus, puis exec() pour charger le nouveau code binaire.

- Arborescence PID / PPID : Filiation stricte des processus remontant jusqu'au processus racine systemd ou init (PID 1).
- Appel système fork() : Duplique à l'identique le processus parent pour engendrer un processus enfant avec son propre PID.

• Appel système exec() : Remplace le code et l'espace mémoire d'un processus par un nouveau programme binaire à exécuter.

Contrôle des Tâches et Communication Inter-Processus : Les Signaux POSIX

Les signaux constituent la forme de communication inter-processus (IPC) la plus élémentaire sous Unix. Lorsqu'un signal est transmis à un processus cible, le noyau interrompt le déroulement séquentiel des instructions pour forcer le traitement du signal. Par défaut, la plupart des signaux entraînent la terminaison du processus cible. Toutefois, un développeur peut capturer un signal et lui associer une routine personnalisée (signal handler). Cependant, pour préserver la sécurité du système, les signaux SIGKILL (9) et SIGSTOP (19) échappent à ce mécanisme : ils sont interceptés et exécutés en direct par le noyau.

- Signal POSIX : Interruption logicielle asynchrone transmise par le noyau à un processus.
- Gestionnaire de signal (Signal Handler) : Routine logicielle pour intercepter (catch) un signal.
- SIGINT (Signal 2) : Émis par Ctrl + C, demande l'interruption douce d'une tâche interactive.
- SIGTERM (Signal 15) : Signal de terminaison propre par défaut de la commande kill.
- SIGKILL (Signal 9) : Ordre d'arrêt inconditionnel géré par le noyau, incapturable et inconfigurable.

Cas concret enseignant : *L'analogie de l'expulsion de l'immeuble : SIGTERM (Signal 15) est le gardien d'immeuble qui frappe poliment à votre porte pour vous demander de quitter les lieux (vous disposez du temps nécessaire pour ranger votre bureau, sauvegarder vos travaux et fermer à clé). SIGKILL (Signal 9) représente l'intervention des forces de sécurité qui défoncent la porte et vous expulsent sur-le-champ sans vous laisser la possibilité de toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal Linux alors que la commande 'kill -9 1234' fonctionne parfaitement ?
Réponse du professeur : La commande kill attend un numéro de signal préfixé par un tiret (comme -9 pour SIGKILL ou -15 pour SIGTERM). Sans le tiret, le shell interprète le premier argument comme un identifiant de tâche (job ID) ou considère la syntaxe comme erronée.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment peut-on le détruire avec la commande kill ?
Réponse du professeur : Un processus zombie est un processus qui a terminé son exécution mais dont l'entrée est toujours présente dans la table des processus du noyau car le père n'a pas lu son code de retour (appel wait()). Il est impossible de tuer un zombie avec un signal kill car il est déjà mort ; la solution consiste à éliminer son processus parent.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un administrateur effectue la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle exact du symbole '&', détaillez ce qui se passe lors de l'exécution de la commande kill -15 et décrivez le résultat de la vérification par ps.

Démarche de résolution :
Étape 1 : Le symbole '&' placé à la fin de la commande indique au shell d'exécuter la tâche en arrière-plan (background), renvoyant immédiatement la main à l'utilisateur. Étape 2 : La commande ps -ef | grep sleep recherche dans la table des processus actifs la ligne correspondant à sleep pour identifier son PID exact. Étape 3 : La commande kill -15 envoie le signal SIGTERM (Signal 15) au processus, provoquant son arrêt propre.

Résultat : Le processus sleep est stoppé proprement et disparaît de la table des processus actifs lors d'une vérification ultérieure avec ps.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement du système ?

R : systemd (ou init), avec le PID 1.

Q : Quel signal POSIX est généré et transmis au processus au premier plan lorsqu'un utilisateur presse la combinaison 'Ctrl + C' dans son terminal ?

R : Le signal SIGINT (Signal 2).

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté par un gestionnaire de signal écrit par un développeur ?

R : Parce qu'il est géré directement et exclusivement par le noyau du système d'exploitation afin de garantir une issue de secours inévitable en cas de blocage critique.